

🎯 本章目标

掌握 C++ 面向对象的核心机制:封装与访问控制、继承与组合、虚函数多态与虚表、多继承与虚继承、运算符重载。理解每个机制在底层是什么(C 程序员最能接受的视角),并能在工程中正确选择继承、组合与接口。

📦 前置知识

熟悉结构体与函数指针(第1、4章);了解构造函数与析构函数、this 指针(第4、5章)。本章代码基于 C++17,多态部分需要智能指针配合(第5章)。

§6.1 封装与访问控制	§6.2 继承与组合	§6.3 虚函数与虚表
§6.4 多继承与虚继承	§6.5 运算符重载	§6.6 设计对比与取舍

§6.1 封装与访问控制

类的封装有两层含义:数据与操作绑定在一起(成员函数直接操作自己的成员),外部只能通过公开接口访问内部状态。C++ 用三个关键字控制访问:public 任何代码可访问;private 只有本类的成员函数可访问;protected 本类与派生类可访问。struct 与 class 的唯一区别是默认访问级别:struct 默认 public,class 默认 private——这是 C 程序员最容易踩的第一个坑。

封装的工程价值在修改成本:内部实现怎么改都不影响调用方,只要公开接口不变。C 的结构体把所有字段暴露给所有人,改一个字段名,全部使用点全部要改;C++ 把字段放进 private,外部只能通过接口访问,内部从裸数组换成 vector 都无人察觉。const 成员函数(第4章)把"只读操作"也写进了接口,封装与不变式一起构成类的契约。

C 的写法	C++ 的写法	说明
struct Student { char name[32]; double score; };	class Student { private: ... public: ... };	C 字段全裸,C++ 默认私有强制走接口
全局函数操作结构体	成员函数直接访问私有字段	C++ 操作与数据同处一个作用域
改字段名牵连全部调用点	改实现不影响调用方	封装隔离变化,降低维护成本

清单 6-1 封装与访问控制

```

#include <iostream>
#include <vector>

class BankAccount {
public:
    explicit BankAccount(double init) : balance(init) {}

    void deposit(double v) {
        if (v > 0) balance += v;          // 不变式:余额永远非负
    }

    bool withdraw(double v) {
        if (v < 0 || v > balance) return false;    // 拒绝透支
        balance -= v;
        return true;
    }

    double getBalance() const { return balance; }    // 只读接口

private:
    double balance;                                // 私有:外部无法直接修改
};

int main() {
    BankAccount acc(100);
    acc.deposit(50);
    if (acc.withdraw(200)) std::cout << "取款成功\n";
    else std::cout << "余额不足:" << acc.getBalance() << "\n";
    // acc.balance = -999;                        // 错误:private 不可访问
    return 0;
}

```

§6.2 继承与组合

继承建立"is-a"关系:派生类是一个基类,自动拥有基类的公开接口,并可以扩展或改写。组合建立"has-a"关系:一个类持有另一个类的对象作为成员。经验法则:能组合就不继承——组合更灵活,运行期可替换,没有继承带来的耦合;只有语义上确定"派生类确实是一种基类",并且需要多态(见下节)时才用继承。判断方法叫 Liskov 替换:任何使用基类的地方,换成派生类行为都不应出错。

构造与析构的顺序是继承的基础纪律:构造时先基类后成员最后自己的构造体,析构时严格相反。这意味着派生类的构造函数在基类构造完成之前不能访问基类成员——初始化列表里的基类初始化先于一切。C 没有继承语法,老代码用"结构体里放结构体"模拟,但类型之间没有"is-a"关系,也无法多态,只是数据布局上的巧合。

C 的写法	C++ 的写法	说明
struct Manager { struct Employee base; ... };	class Manager : public Employee { ... };	C 是组合模拟,C++ 有真正的 is-a 关系
mgr.base.name 访问基类字段	mgr.name 直接访问	C++ 基类成员对派生类无缝可见
无法把 Manager* 当 Employee* 用	Manager* 隐式转换为 Employee*	C++ 派生类指针可当基类指针用

清单 6-2 继承的构造析构顺序

```
#include <iostream>

class Base {
public:
    Base() { std::cout << "1 基类构造\n"; }
    virtual ~Base() { std::cout << "3 基类析构\n"; }
};

class Member {
public:
    Member() { std::cout << "2 成员构造\n"; }
    ~Member() { std::cout << "4 成员析构\n"; }
};

class Derived : public Base {
public:
    Derived() : Base(), m() { std::cout << "5 派生类构造\n"; }
    ~Derived() override { std::cout << "6 派生类析构\n"; }
private:
    Member m;          // 组合成员
};

int main() {
    Derived d;          // 输出顺序:1 基类 → 2 成员 → 5 派生类
    return 0;           // 析构顺序:6 → 4 → 3,与构造完全相反
}
```

§6.3 虚函数与虚表:多态的实现

多态的核心需求:通过基类指针调用函数,实际执行的是派生类的版本。C++ 用 `virtual` 关键字声明虚函数,调用时按对象的实际类型动态绑定。机制是虚表:每个含虚函数的类编译期生成一张虚函数表(vtable),对象里藏一个指向虚表的指针(vptr);通过基类指针调用虚函数时,编译器生成"取 vptr → 查表 → 间接调用"的代码,实际调用哪个函数由对象自己决定。这与 C 里 FILE 结构体的函数指针表是同一个套路,只不过 C++ 是编译器自动生成。

三条配套纪律。第一,基类析构函数必须是 `virtual`:通过基类指针 `delete` 派生对象时,只有析构是虚函数才会先调用派生类析构再调用基类析构,否则派生类部分漏析构,资源泄漏。第二,C++11 起用 `override` 标记重写,用 `final` 禁止继续派生,编译器会检查 `override` 是否真的重写了基类虚函数——拼错签名立刻报错。第三,纯虚函数让类变成抽象类:抽象类不能实例化,只能作为接口与骨架,派生类必须实现全部纯虚函数才能创建对象。

C 的写法	C++ 的写法	说明
<code>struct FileOps { int (*read)(...); ... };</code>	<code>class Stream { virtual int read(...) = 0; };</code>	C 手写函数指针表,C++ 虚表自动生成
<code>obj->ops->read(obj, buf, n)</code>	<code>obj->read(buf, n)</code>	C 显式传对象,C++ 隐式 this
换实现要手工改函数指针表	继承 + <code>override</code> 即可	C++ 扩展新类型不改既有代码

清单 6-3 虚函数、纯虚函数与虚析构

```

#include <iostream>
#include <memory>
#include <vector>

class Shape {                                // 抽象类:含纯虚函数
public:
    virtual ~Shape() = default;              // 虚析构:多态删除的安全保障
    virtual double area() const = 0;         // 纯虚函数:必须由派生类实现
};

class Circle final : public Shape { // final:不许再被继承
public:
    explicit Circle(double r) : r(r) {}
    double area() const override { return 3.14159 * r * r; }
private:
    double r;
};

class Rect : public Shape {
public:
    Rect(double w, double h) : w(w), h(h) {}
    double area() const override { return w * h; }
private:
    double w, h;
};

int main() {
    std::vector<std::unique_ptr<Shape>> shapes;
    shapes.push_back(std::make_unique<Circle>(1.0));
    shapes.push_back(std::make_unique<Rect>(3.0, 4.0));
    for (const auto& s : shapes) { // 同一接口,各自实现
        std::cout << "面积:" << s->area() << "\n";
    }
    return 0;
}

```

⚠ 易错点 1:析构函数不是虚函数

class Base 有虚函数却写普通析构,通过 Base* delete 派生对象时,只有 Base 的析构被调用,派生类的成员与资源不释放,悄悄泄漏。规则一句话:类里有虚函数,析构函数就必须是 virtual,这是编译器无法替你检查的纪律。

⚠ 易错点 2:构造函数里调用虚函数不会多态

构造派生类时,基类构造阶段虚表还指向基类,调用虚函数执行的是基类版本;析构时相反。想在构造期间完成"虚初始化"是错的,应该用工厂函数或两阶段构造,不要在构造/析构函数里依赖虚调用。

§6.4 多继承与虚继承

一个类可以有多个基类,这叫多继承。多继承最常见的用途是"接口式继承":一个实现类同时继承一个功能基类和几个纯接口(如 Drawable、Serializable),获得多方面的能力。风险在菱形继承:类 A 有数据,B 和 C 都继承 A,D 同时继承 B 和 C,那么 D 里会有两份 A 的子对象,成员访问产生歧义,数据冗余,语义混乱。

解决办法是虚继承:把 A 声明为虚基类(virtual public A),B 与 C 共享同一份 A 子对象,菱形变成共享的三角。虚继承有代价:虚基类子对象的地址需要运行时偏移计算,对象内存布局复杂,构造时虚基类只能由最派生类直接初始化。因此虚继承只在"确实需要共享基类状态"时使用;日常编码用接口式多继承 + 组合,能避开大多数菱形场景。

清单 6-4 菱形继承与虚继承修复

```
#include <iostream>
#include <string>

class Person {                      // 虚基类:共享唯一子对象
public:
    explicit Person(std::string n) : name(n) {}
    std::string name;
};

class Student : virtual public Person { // 虚继承
public:
    Student(std::string n, double g)
        : Person(n), grade(g) {}
    double grade;
};

class Teacher : virtual public Person { // 虚继承
public:
    Teacher(std::string n, int y)
        : Person(n), years(y) {}
    int years;
};

// 既是学生又是教师:D 中只有一份 Person
class TeachingAssistant : public Student, public Teacher {
public:
    TeachingAssistant(std::string n, double g, int y)
        : Person(n), Student(n, g), Teacher(n, y) {}
    void who() const {
        std::cout << name << ": 成绩 " << grade
                    << ", 教龄 " << years << " 年\n";
    }
};

int main() {
    TeachingAssistant ta("张三", 92.0, 3);
    ta.who();                      // name 无歧义:虚继承保证唯一
    return 0;
}
```

§6.5 运算符重载

运算符重载让自定义类型使用与内置类型一致的表达式语法: $a + b$ 、 $v[i]$ 、 $p \rightarrow x$ 。C++ 允许重载绝大多数运算符,但有几条铁律:不能发明新运算符(如 $**$),不能改变优先级与结合性,不能改变操作数个数;重载后不保证"短路的 $\&\&$ 、 $\|\|$ "与求值顺序的语义——所以工程上严禁重载逻辑与、逻辑或和逗号运算符。比较运算符最常见,重载 $==$ 时通常配套重载 $!=$,否则两个运算符语义不一致。

实现形态有两种:成员函数重载,左操作数必须是本类对象(如 $a + b$ 中 a 是本类);非成员函数重载,适合左操作数是内置类型或需要隐式转换的场景。输出运算符与输入运算符(左操作数是流)只能写成非成员函数,且要声明为类的 friend 才能访问私有成员。friend 的意义是"指定外部函数为友",它只是破例放行,不是反向的 public——应保持最小化。

C 的写法	C++ 的写法	说明
Point add_point(p, q);	Point r = p + q;	C 手写函数名,C++ 表达式自然可读
int eq_point(p, q);	if (p == q) ...	C 靠命名约定,C++ 运算符语义内建
printf("%d,%d", p.x, p.y)	std::cout << p	C++ 重载输出流运算符,格式随类型

清单 6-5 运算符重载:复数类

```
#include <iostream>

class Complex {
public:
    Complex(double r = 0, double i = 0) : re(r), im(i) {}

    Complex operator+(const Complex& o) const {    // 成员重载 +
        return Complex(re + o.re, im + o.im);
    }
    Complex operator-() const { return Complex(-re, -im); } // 一元负号
    bool operator==(const Complex& o) const {
        return re == o.re && im == o.im;
    }

    friend std::ostream& operator<<(std::ostream& os, const Complex& c);

private:
    double re, im;
};

std::ostream& operator<<(std::ostream& os, const Complex& c) {
    return os << c.re << (c.im >= 0 ? "+" : "") << c.im << "i";
}

int main() {
    Complex a(1, 2), b(3, 4);
    Complex c = a + b;           // 调用 operator+
    std::cout << "a+b=" << c << "\n";           // 输出 4+6i
    std::cout << "-a=" << -a << "\n";           // 输出 -1-2i
    std::cout << (c == Complex(4, 6) ? "相等" : "不等") << "\n";
    return 0;
}
```

 **易错点 3:重载运算符破坏直觉语义**

重载 && 和 || 会丢失短路求值与求值顺序,重载逗号运算符会打乱表达式语义,这些运算符被禁得越远越好;重载 == 却不重载 !=、重载 < 却不重载 > 也让调用方困惑。原则:重载运算符必须保持内置运算符的直觉语义,拿不准就不重载,改用命名函数。

§6.6 设计对比与取舍

把 C 与 C++ 的面向对象放在一起看:数据 + 函数指针表就是 vtable 的雏形,结构体嵌套就是继承的雏形,显式传对象指针就是 this 的雏形——C++ 的全部机制都能在 C 里找到手工版本,只是编译器接管了样板代码与安全检查。差别在于:C 的模拟是约定,靠程序员自觉;C++ 是语言规则,靠编译器强制。同样的代码,C 的版本漏改一处就出 bug,C++ 的版本编译期就拒绝。

工程上的取舍建议:接口(纯虚函数)定义能力边界,功能基类承载通用实现,组合表达依赖关系,虚继承只用于真正的共享菱形。多态容器用 unique_ptr 或 shared_ptr 持有基类指针(第5章),析构顺序与虚析构配合,内存与对象模型两不误。面向对象不是目的,可维护性才是;能用值语义与算法解决的需求,不必硬造继承树。

面向对象速查表

概念	语法示例	要点
封装	private 字段 + public 接口	实现与接口分离,变化隔离
继承	class D : public B	is-a 关系,自动获得基类接口
组合	class D { B b; }	has-a 关系,优先于继承
虚函数	virtual double area();	动态绑定,运行时按对象类型分派
虚析构	virtual ~Base() = default;	基类有虚函数就必须有虚析构
override	double area() override;	编译器核对确实重写了虚函数
final	class Circle final : public Shape	禁止继承或继续重写
纯虚函数	virtual void f() = 0;	抽象类不可实例化,派生类必须实现
抽象类	class Shape { virtual ... = 0; }	定义接口,不做实现
多继承	class D : public B, public C	接口式多继承,警惕菱形
虚继承	class B : virtual public A	菱形共享唯一子对象
运算符重载	Complex operator+(const Complex&)	保持直觉语义,不重载 && 与逗号
friend	friend std::ostream& operator<<(...)	指定友元,只给必要的函数放行
虚表 vtable	编译器自动生成	虚函数查表间接调用,略有开销

自测题

- Q 1. struct 与 class 的区别是什么?为什么封装能降低维护成本?
- Q 2. 继承与组合分别表达什么关系?为什么"能组合就不继承"?
- Q 3. 虚函数多态的底层机制是什么?为什么基类析构必须声明为 virtual?
- Q 4. 菱形继承有什么问题?虚继承如何解决?虚继承有什么代价?
- Q 5. 运算符重载有哪些铁律?为什么严禁重载逻辑与和逻辑或?

✅ 参考答案

1. 两者唯一的区别是默认访问级别:struct 默认 public,class 默认 private。封装把实现细节关进私有区,内部改动只要接口不变,调用方零修改;同时不变式(如余额非负)由成员函数统一维护,外部无法绕过。
2. 继承是 is-a(派生类是一种基类),组合是 has-a(对象内部持有另一个对象)。组合耦合更弱、可运行期替换,继承则把派生类与基类实现绑定,改动基类常波及所有派生类。语义上不是 is-a 就不该继承。
3. 每个含虚函数的类有一张虚表,对象携带虚表指针;虚调用是"取指针、查表、间接跳转"。基类析构若非虚,通过基类指针 delete 派生对象只析构基类部分,派生类资源泄漏;声明为 virtual 后删除按实际类型走完整的析构链。
4. 菱形中 D 继承 B 与 C,而 B、C 各有 A 的子对象,D 中就有两份 A 的成员,访问歧义且数据重复。虚继承让 B、C 共享同一份 A 子对象,歧义消失;代价是虚基类偏移需运行时计算,且虚基类构造只能由最派生类负责。
5. 不能发明新运算符、不能改优先级与结合性、不能改操作数个数;重载后的逻辑与、逻辑或不再短路,求值顺序也不保证,违背内置语义,工程上直接禁用;重载 == 要配套 !=,输出运算符要返回流引用。

实验:图形系统

本实验把本章机制用在一个完整的图形系统上:抽象基类定义接口,派生类实现面积计算,接口式多继承让图形可绘制,运算符重载让点支持加减与相等比较,多态容器统一管理。代码分三部分,覆盖点已在注释中标明。

实验 6•第一部分:抽象基类与多继承(覆盖:纯虚函数、虚析构、override、多继承)

```

#include <iostream>
#include <string>
#include <vector>
#include <memory>

class Shape {                                // 覆盖:抽象类 + 纯虚函数
public:
    virtual ~Shape() = default;              // 覆盖:虚析构,多态删除安全
    virtual double area() const = 0;
};

class Drawable {                             // 覆盖:接口式基类(纯接口)
public:
    virtual ~Drawable() = default;
    virtual void draw() const = 0;
};

class Rect : public Shape {
public:
    Rect(double w, double h) : w(w), h(h) {}
    double area() const override { return w * h; }
protected:
    double w, h;
};

// 覆盖:多继承——同时是形状与可绘制物
class Square : public Rect, public Drawable {
public:
    explicit Square(double side) : Rect(side, side) {}
    double area() const override { return w * w; }
    void draw() const override {
        std::cout << "绘制正方形,边长 " << w << "\n";
    }
};

class Circle : public Shape, public Drawable {
public:
    explicit Circle(double r) : r(r) {}
    double area() const override { return 3.14159 * r * r; }
    void draw() const override {
        std::cout << "绘制圆形,半径 " << r << "\n";
    }
private:
    double r;
};

```

实验 6·第二部分:运算符重载(覆盖:成员重载、一元运算符、friend 输出)

```

class Point {
public:
    Point(double x = 0, double y = 0) : x(x), y(y) {}

    Point operator+(const Point& o) const {          // 成员重载 +
        return Point(x + o.x, y + o.y);
    }
    Point operator-(const Point& o) const {          // 成员重载 -
        return Point(x - o.x, y - o.y);
    }
    Point operator*(double k) const {                // 数乘
        return Point(x * k, y * k);
    }
    bool operator==(const Point& o) const {          // 相等比较
        return x == o.x && y == o.y;
    }
    double norm() const { return x * x + y * y; }

    friend std::ostream& operator<<(std::ostream& os, const Point& p);

private:
    double x, y;
};

std::ostream& operator<<(std::ostream& os, const Point& p) {
    return os << "(" << p.x << ", " << p.y << ")";
}

```

实验 6•第三部分:多态容器与组合(覆盖:虚表动态绑定、unique_ptr 多态、组合)

```

int main() {
    // 覆盖:多态容器—基类指针持有不同派生对象
    std::vector<std::unique_ptr<Shape>> shapes;
    shapes.push_back(std::make_unique<Rect>(3.0, 4.0));
    shapes.push_back(std::make_unique<Square>(2.0));
    shapes.push_back(std::make_unique<Circle>(1.0));

    double total = 0.0;
    for (const auto& s : shapes) {
        total += s->area();           // 覆盖:虚函数动态绑定
        std::cout << "面积 " << s->area() << "\n";
    }
    std::cout << "总面积 " << total << "\n";

    // 覆盖:多继承对象的双接口使用
    Square sq(3.0);
    Shape& asShape = sq;              // 向上转型:is-a
    Drawable& asDraw = sq;
    std::cout << "正方形面积 " << asShape.area() << "\n";
    asDraw.draw();

    // 覆盖:运算符重载综合
    Point a(1, 2), b(3, 4);
    Point c = a + b - Point(1, 1);
    std::cout << c << (c == Point(3, 5) ? " 验证通过" : " 验证失败") << "\n";
    return 0;
}

```

实验覆盖点核对:封装(BankAccount 思路在 §6.1 已练)、继承与组合(Shape/Rect/Square 链与 shapes 容器)、虚函数与虚表(area 的动态绑定)、虚析构(基类声明,智能指针自动调用)、override 与 final、纯虚函数与抽象类(Shape/Drawable)、多继承(Square 同时继承 Rect 与 Drawable)、运算符重载(Point 的加减乘、相等、输出)、friend(输出运算符)。自行扩展:给 Point 增加 < 运算符后放入 std::map,或给 Circle 增加放大函数,体会"改一个类不动其他代码"的多态红利。

工程建议 1:接口与实现分离

定义功能的抽象基类只放纯虚函数,不放数据成员;实现细节全在派生类。这样新增一种图形只需写一个新类,既有代码零修改——开闭原则的直接落地,也是多态价值的最大化。

工程建议 2:多态对象交给智能指针

多态对象一律用 unique_ptr 或 shared_ptr 持有,容器只放指针:对象大小不同,值语义无法装进同一个 vector。配合虚析构与 make_unique,释放、扩容、异常路径全部自动,内存模型与对象模型互不打架。

本章小结

面向对象四件套:封装把实现关进私有区,继承与组合表达类型关系,虚函数用虚表实现动态绑定,运算符重载让自定义类型拥有自然语法。多继承与虚继承解决接口扩展与菱形共享,但日常优先组合与接口。下一章讲文件 I/O:从 C 的 FILE 到 C++ 的流与文件系统。